

Afformed Campus Recruitment Assessment

Detailed Problem Statement & Technical Analysis

Assessment Overview

Duration: 3 Hours

Candidates must choose one track:

- Full Stack
- Backend
- Frontend

The assessment evaluates:

- REST API Development
- Data Structures & Algorithms
- Object-Oriented Programming
- API Integration
- React Development
- Git & GitHub Workflow
- Software Engineering Best Practices

The objective is not merely to write code but to design scalable, maintainable, and production-quality solutions within a limited time.

Problem 1: Number Management Service

Objective

Develop a REST API microservice named:

GET /numbers

The API accepts multiple URLs through query parameters.

Example:

/numbers?url=<http://server/primes>&url=<http://server/fibo>&url=<http://server/odd>

Each URL returns:

```
{
  "numbers": [1,2,3,5,8,13]
}
```

Expected Processing

The service must:

1. Validate URLs
2. Fetch data from all valid URLs
3. Ignore invalid URLs
4. Ignore slow responses
5. Merge all received numbers
6. Remove duplicates
7. Sort in ascending order
8. Return final result

Example Response:

```
{
  "numbers": [1,2,3,5,7,8,9,11,13]
}
```

Skills Evaluated

Backend Development

- Express.js / Spring Boot / Flask
- REST API design
- Query parameter handling

Concurrency

Candidates should ideally use:

- Promise.all()
- Async/Await
- Parallel API execution

Data Structures



- HashSet
- Arrays
- Sorting Algorithms

Performance Engineering

A strict timeout requirement of 500 milliseconds exists.

Candidates must:

- Handle slow APIs
- Prevent blocking execution
- Return results quickly

Error Handling

Candidates must gracefully handle:

- Invalid URLs
- Network failures
- Timeout exceptions

Difficulty Level

Medium

Industry Relevance: High

Problem 2: Prefix Management Service

Objective

Develop a REST API:

GET /prefixes

Example:

/prefixes?keywords=bonfire,bool

Server contains predefined words:

```
[  
  "bonfire",  
  "cardio",  
  "case",  
  "character",  
  "bonsai"  
]
```

Expected Processing

For each keyword:

1. Check existence
2. If found:
Generate smallest unique prefix
3. If not found:
Return not_found

Example:

```
[  
  {  
    "keyword": "bonfire",  
    "status": "found",  
    "prefix": "bonf"  
  },  
  {  
    "keyword": "bool",  
    "status": "not_found",  
    "prefix": "not_applicable"  
  }  
]
```

Skills Evaluated

String Processing

- Prefix comparison
- Character matching

Algorithm Design

Candidates must determine:



"What is the shortest prefix that uniquely identifies a word?"

Data Structures

Possible solutions:

- Trie
- Nested loops
- HashMap-based prefix counting

Time Complexity Awareness

Efficient solutions are preferred.

Difficulty Level

Medium

Industry Relevance: Medium to High

Full Stack Assessment

Question 1

Build a Product Aggregation Microservice.

Features

1. Register with external API
2. Generate authentication token
3. Consume product APIs from multiple vendors
4. Aggregate product information
5. Apply sorting
6. Apply filtering
7. Apply pagination
8. Expose public APIs

Skills Evaluated

- REST APIs
- JWT Authentication
- API Integration
- Pagination
- Sorting
- Aggregation Logic
- Backend Architecture



Question 2

Build a React Frontend.

Requirements

- React / Next.js
- Mobile Responsive Design
- Desktop Responsive Design
- Consume APIs from Question 1
- No direct calls to test server

Skills Evaluated

- React Fundamentals
- API Consumption
- Responsive Design
- State Management
- Component Architecture
- UI Engineering

Overall Assessment Difficulty

Backend Track: Medium

Frontend Track: Medium

Full Stack Track: Medium-High

Primary Focus:

Engineering Implementation + API Integration + Practical Problem Solving

Secondary Focus:

DSA and OOP Concepts